CityMind AI

One neural network that runs 98 traffic lights - and does better than the simulator's own signal logic

THE PROJECT IN 60 SECONDS

1. **Problem.** Every traffic light needs a rule for when to turn green. We simulate the morning rush hour (07:00-08:00, about 15,000 cars, 98 signalised junctions) of the German city Ingolstadt in the traffic simulator SUMO. The simulator's own rule - "actuated" control, which keeps a green on while cars keep arriving - is the opponent we must beat.
2. **Idea.** Let a computer *learn* the rule by trial and error (reinforcement learning). We train **one** neural network and copy it to all 98 junctions. Every 5 simulated seconds each junction shows the network 22 numbers about its traffic and gets back one answer: *stay green* or *move to the next green*.
3. **Learning.** The network is rewarded for fewer waiting cars. It practised on about 11 million such decisions using the PPO algorithm (book Ch.11), with a few project-specific fixes that made the difference between "learns nothing" and "wins".
4. **Result.** Tested on 10 random traffic seeds against the baseline on identical traffic, both trained models are better on 12 / 11 of 14 measures and worse on none. Cars wait about **41 % less** and about **200-260 more trips finish per hour**.

-41 %
mean waiting time per car
6.39 s → 3.77 s (model A)

-15 %
lag per scheduled car
276.8 s → 235.4 s

-28 %
mean queue length
0.440 → 0.318 cars/lane

+258
trips completed per hour
11,585 → 11,842

Measure (one simulated hour, mean of 10 seeds)	SUMO actuated (baseline)	Model A	Model B	Verdict (both models)
Lag per scheduled car, s ↓	276.8	**235.4**	**239.9**	better on 10 / 10 seeds
Mean waiting time, s ↓	6.39	**3.77**	**3.86**	better on 10 / 10 seeds
Backlog at 08:00, cars ↓	3,108	**2,852**	**2,921**	better on 10 / 10 seeds
Mean speed, m/s ↑	8.70	**9.37**	**9.30**	better on 10 / 10 seeds
Trips completed ↑	11,585	**11,842**	**11,791**	better on 10 / 10 seeds
Cars that never got into the network ↓	503	501	481	**level - not a win** (5 / 10 and 7 / 10 seeds)
Teleports (gridlock events) ↓	13.3	7.5	10.0	lower on average, but too noisy to call (6 / 10 seeds)

All 14 measures are in section 10. ↓ = lower is better, ↑ = higher is better. Model A and model B are two independent trainings (different random seeds).

HOW TO READ THIS REPORT

The report explains the project the way a textbook would: first the vocabulary of reinforcement learning (agent, state, action, reward, policy), then how each word is realised in our code, then the learning algorithm, then the evidence.

- BOOK Ch.3 p.44 a green tag = the concept is explained in *Hands-On Reinforcement Learning with Python* by Sudharsan Ravichandiran (Packt, 2018), "the book". Page numbers are the printed ones.
- `path/file.py:85` a grey tag = the line of project code that implements it (links open the file in the project repository).
- Orange boxes mark things that are **not** in the book or that are honest limits of our results.

1 Map: book concept → what we did → where it is in the code

This table is the index of the whole report. Each row is explained in the section named in the last column.

Concept	In this project	Book	Code	Sect.
Agent, environment, reward	Shared neural network ↔ SUMO traffic simulation	BOOK Ch.1 p.9-11	`src/agents/ippo_agent.py:151` `src/environment/decentralized_sumo_env.py:43`	3
State, Markov property, partially observable	22 numbers per junction (own lanes + neighbours)	BOOK Ch.3 p.41 BOOK Ch.1 p.13	`src/environment/local_observation.py:151`	4
Action (discrete)	0 = keep green, 1 = go to next green, behind a safety layer	BOOK Ch.1 p.11	`src/environment/safety.py:106`	5
Reward, return	Minus the cars waiting at and around the junction	BOOK Ch.3 p.44	`src/environment/local_reward.py:85`	6
Reward clipping	Cost capped at 200, then scaled by 0.1	BOOK Ch.8 p.174	`src/environment/local_reward.py:97`	6
Discount factor γ	γ = 0.99 (about 8 minutes of look-ahead)	BOOK Ch.3 p.44-45	`configs/ippo.yaml:12`	3
Policy and value function	Actor network and critic network, 22-256-256 each	BOOK Ch.3 p.45-46 BOOK Ch.1 p.9-10	`src/agents/ippo_agent.py:221`	7
Neural network, activation, gradient descent	tanh units, Adam optimiser, learning rate 3e-4	BOOK Ch.7 p.132 BOOK Ch.7 p.136	`configs/ippo.yaml:25`	7

Policy gradient	Make actions that led to high return more likely	BOOK Ch.11 p.225-226	(inside PPO)	7
Actor-critic, advantage = Q - V	Critic scores states, actor picks actions	BOOK Ch.10 p.209-211	`src/agents/ippo_agent.py:221`	7
Entropy bonus	c_2 = 0.01 keeps the policy from freezing early	BOOK Ch.10 p.211	`configs/ippo.yaml:16`	7
TRPO → PPO, clipped objective	The training algorithm, ε = 0.2	BOOK Ch.11 p.240 BOOK Ch.11 p.245-247	`src/agents/ippo_agent.py:221`	7
Many environments in parallel	6 SUMO simulations feed one learner	BOOK Ch.10 p.209	`src/agents/ippo_agent.py:37`	7
Generalised advantage estimation (GAE)	λ = 0.95	**not in book**	`configs/ippo.yaml:13`	7
One network for all junctions (IPPO)	Parameter sharing: 98 junctions = 98x the experience	**not in book**	`src/agents/ippo_agent.py:37`	7
Split gradient clipping	The fix that made the network learn at all	**not in book**	`src/agents/split_clip.py:28`	8
Drain-aware metrics, paired statistics	Lag per scheduled car, sign test over 10 seeds	**not in book**	`src/evaluation/drain_metrics.py:387` `scripts/report_eval.py:67`	9

2 The problem and the opponent

The scenario

We use **InTAS**, a published, realistic model of Ingolstadt's road network and traffic, inside the open-source simulator **Eclipse SUMO**. We simulate one morning rush hour, 07:00-08:00, in which about 15,000 cars are scheduled to drive. **98 junctions** have traffic lights. The simulator moves every car second by second, so a bad signal rule shows up as queues, slow trips and cars that cannot even enter the network because the entry road is jammed.

The opponent: SUMO "actuated" control

76 of the 98 signal programs in the scenario are *vehicle-actuated*: a detector sees cars, the green is extended while cars keep coming and ends at a gap, always between a minimum and a maximum duration. This is a good, adaptive rule - far better than a plain timer. We confirmed it: a pure fixed-time plan is **worse** than actuated control on 11 of the 14 measures (lag 298.7 s vs 276.8 s), so actuated control is the real bar.

WHERE IN THE CODE

Baseline controllers: `src/controllers/sumo_actuated.py` (lets SUMO run its own programs untouched), `src/controllers/fixed_time.py` (plain timer, for context). Trained agent: `src/evaluation/evaluate.py:257` (`_run_ippo_episode`). All three are scored by the same code.

What "better" means - the goal in numbers

We do not want a model that looks good only on one number. We therefore score **14 measures** (section 10), chosen so that the model cannot win by cheating - for example, by blocking cars from entering the network (that would shorten queues inside but leave cars waiting outside). The main number is **lag per scheduled car**: for every car that was *supposed* to drive in this hour, how many seconds later than on an empty road did it finish? Cars that never finish or never enter count as "late until 08:00". Section 9 gives the formula.

---

3 Reinforcement learning, in our words

WHAT THE BOOK SAYS

RL is learning by trial and error: an **agent** acts in an **environment**, sees the new **state** and gets a **reward**; its goal is to maximise the total reward (BOOK Ch.1 p.6-11). The agent's behaviour is its **policy**; the **value function** says how good a state is under that policy (BOOK Ch.1 p.9-10, BOOK Ch.3 p.45-46). The problem is formalised as a **Markov decision process** (BOOK Ch.3 p.43).

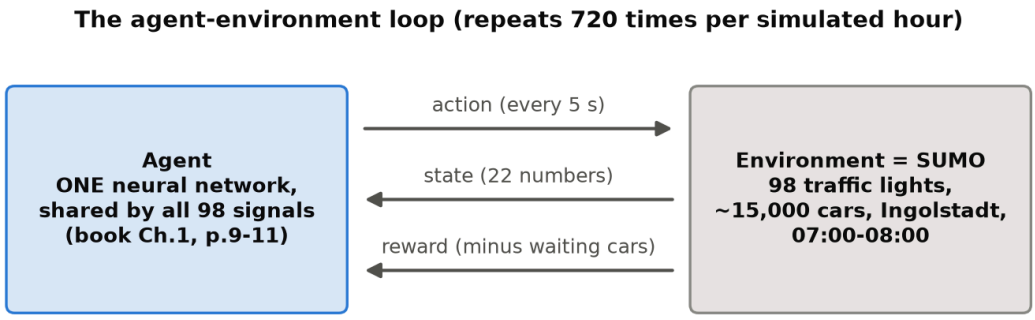


Figure 1 - The agent-environment loop of BOOK Ch.1 p.11, with our real quantities. One pass = 5 simulated seconds; one hour = 720 passes.

RL word	In CityMind	Code
Environment	SUMO simulating Ingolstadt; one agent slot per traffic light (98), wrapped in the standard multi-agent interface PettingZoo.	`src/environment/decentralized_sumo_env.py:43`
Agent	One neural network that is *copied* to every junction - all junctions share the same weights.	`src/agents/ippo_agent.py:151`
State s	22 numbers describing one junction and its neighbours (section 4).	`src/environment/local_observation.py:151`
Action a	Two choices, every 5 s: `0` keep the current green, `1` advance to the next green.	`src/environment/safety.py:106`
Reward r	Minus (weighted) cars waiting - always ≤ 0, so "less negative" = better (section 6).	`src/environment/local_reward.py:85`

Policy π(a|s)	The actor network: 22 inputs → 256 → 256 → probability of "advance".	`src/agents/ippo_agent.py:221`
Value V(s)	The critic network (same shape): predicts how much waiting is still to come from this state.	`src/agents/ippo_agent.py:221`
Episode	One simulated hour = 720 decisions per junction.	`configs/environment.yaml:5`

Why it is a hard version of RL

- **Partially observable.** The book's Markov property says the future depends only on the present state (BOOK Ch.3 p.41). A junction cannot see the whole city, only its own lanes and a few neighbours - this is the "partially observable environment" of BOOK Ch.1 p.13 . So we design the 22 numbers carefully to include what matters (section 4).
- **Many agents at once.** 98 junctions act simultaneously and affect each other (a queue released upstream arrives downstream). The book names multi-agent environments among the environment types, as mostly stochastic (BOOK Ch.1 p.13). Our answer is *independent learners with shared weights* (IPPO, section 7).
- **Delayed consequences.** A green given now empties a queue over the next minute; a green given to the wrong road is felt only later. That is the reason for the discount factor below.

Return and the discount factor

The **return** is the total future reward, with later rewards counting a bit less (BOOK Ch.3 p.44-45):

RETURN (BOOK CH.3 P.44)

$$G_t = r_{t+1} + \gamma\, r_{t+2} + \gamma^2\, r_{t+3} + \dots \quad \text{with } \gamma = 0.99$$

With γ = 0.99 a reward 100 steps ahead (100 x 5 s ≈ 8 minutes) still counts about 37 %. That is the time scale on which a queue forms and clears, so it is the right horizon.

WHERE WE DIFFER FROM THE BOOK

The book calls a discount factor "optimal" between 0.2 and 0.8 (BOOK Ch.3 p.45); that is a rule of thumb for its small examples where reward comes quickly. In traffic, the effect of a decision is spread over minutes, so such a small γ would make the agent short-sighted ($0.8^{10} \approx 0.1$: it would ignore anything beyond ~1 minute). 0.99 is also the usual default for PPO.

WHERE IN THE CODE

Loop and one environment step: `src/environment/decentralized_sumo_env.py:191` - runs 5 simulated seconds through the safety layer, then builds new states and rewards for all 98 junctions. Episode length and decision interval: `configs/environment.yaml:6` , `configs/environment.yaml:9` . Fake-SUMO unit tests: `tests/test_envs_fake_traci.py` .

4 State: what each junction sees (22 numbers)

> **IN PLAIN WORDS**
> Imagine a police officer at one junction. To decide "keep green or switch?" they need: *which road has green now, for how long, and how many cars are queuing on each road - including the road behind the short stretch just before the line, cars still waiting to enter the city, and the queue at the next junctions up and down the road.* That is exactly the 22 numbers, each squashed into the range −1 to +1 so the network can digest them.



Nos.	What	How it is computed / why it is there
1-6	Which green is on	One-hot over up to 6 green "slots" (a junction has 2-6 greens). Tells the network which of its own roads currently moves.
7	In transition?	1 while yellow / all-red is running (nothing can be changed then).
8-9	Time in this green	elapsed ÷ *that phase's own minimum green*, and elapsed ÷ its maximum. Scaled into [0,1].
10-15	Demand per green phase	d_i = ∑ over movements that green releases of [halted cars on the lane + halted cars on its feeder lanes + share of waiting-outside cars − halted cars on the lane it flows into]; squashed $d/(\|d\|+8)$.
16	Worst queue	longest queue on the junction's own lanes, q/(q+8).
17	Cars waiting outside	cars scheduled to enter at this junction but not yet inserted, p/(p+20).
18	Number of greens	n_{green} ÷ 6 - tells the shared network what kind of junction it is.
19-22	Neighbours	queue of the 2 strongest road-connected junctions upstream and 2 downstream (0 if none).

SQUASHING FUNCTION (USED FOR 10-22)

$\text{squash}(x) = x / (|x| + k)$ e.g. with $k = 8$: $0 \to 0$ 8 cars $\to 0.50$ 24 cars $\to 0.75$ $-8 \to -0.50$

Any number of cars maps smoothly into $(-1, 1)$; small queues still change the input visibly, huge queues do not saturate abruptly.

Three design decisions that mattered

1. **Timing relative to the phase's own minimum and maximum (inputs 8-9).** We measured which inputs the winning baseline effectively uses: predicting *when actuated control leaves a green* from our old 18 inputs gave an AUC of only 0.757 (0.5 = guessing); adding "elapsed ÷ own minimum green" lifted it to **0.977**. The old agent was blind to the one thing the baseline reacts to. (Analysis: `artifacts/eda/observation_relevance/REPORT.md:35`)
2. **Feeder lanes (reach 300 m).** Some stop-line lanes are only ~20 m long (about 2 cars) while 60 cars wait on the unsignalised road behind them. The old agent saw "2 cars". We now walk upstream along unsignalised roads up to 300 m and add the halted cars. (`src/environment/local_observation.py:191`, setting `queue_reach_m` in `configs/environment.yaml:14`)
3. **Cars waiting outside the network (input 17, and added to demand in stage 2).** A jammed entry road leaves cars queuing *outside* the simulated network, where the agent saw nothing. We count them in input 17 and, in stage 2, add them to the demand of the lane they will queue at (`pending_in_demand`, `configs/environment_pending.yaml:16`).

BOOK LINK

The book's Markov property (BOOK Ch.3 p.41) requires the state to contain what the future depends on. Because a junction cannot see everything (BOOK Ch.1 p.13), these three additions are our way of putting the most decision-relevant hidden facts into the state.

WHERE IN THE CODE

- Layout and names: `src/environment/local_observation.py:33` (`OBS_SIZE`, `OBS_NAMES`); builder: `src/environment/local_observation.py:151` (`build_observation`); per-phase demand: `src/environment/local_observation.py:77`.
- Environment side (queue reach, feeders, road neighbours): `src/environment/decentralized_sumo_env.py:302`, `src/environment/decentralized_sumo_env.py:323`, `src/environment/decentralized_sumo_env.py:340`. Tests: `tests/test_observation.py`.

5 Action: keep or advance - behind a safety layer

IN PLAIN WORDS

The network can only ask one of two things every 5 seconds: "stay on this green" (0) or "move on to the next green" (1). It can never switch a light directly - a safety layer (like a traffic engineer's rule book) translates the request and refuses unsafe ones.

Safety rule	What happens	Code
Minimum green	A request to advance is **ignored** until the current green has run for its own minimum duration.	`src/environment/safety.py:172`
Maximum green	If a green reaches its maximum the layer **forces** the advance, whatever the network says.	`src/environment/safety.py:161`
Yellow / all-red	Every change walks through the signal program's own clearance phases (3 s yellow, 1 s all-red where defined). The network cannot skip them.	`src/environment/safety.py:106`
Own cycle only	"Advance" always goes to the junction's *next green in its own program* - never to an arbitrary phase.	`src/scenario/tls_metadata.py:100`

Per-phase minimum and maximum come from the junction's own authored program when it has them (`_green_bounds`, `src/environment/safety.py:205`); the configured 5 s / 60 s are only fallbacks. Unit and fuzz tests (`tests/test_safety_layer.py`, `tests/test_safety_property.py`) exercise these rules on every real junction.

BOOK LINK

A discrete action space, where the agent picks one of a few actions, is the setting of BOOK Ch.1 p.11 and of the book's policy-gradient chapter (BOOK Ch.11 p.225). Our two-action choice is the simplest case: the policy outputs one probability, p(advance).

HONEST NOTE

The network changes phases more often than the baseline: about 17,600 (A) and 18,600 (B) phase changes per hour network-wide against 14,600 for actuated control. Because every change passes through yellow, this has a cost that the reward charges for (section 6), and the measured results already include it.

6 Reward: what the network is paid for

WHAT THE BOOK SAYS

The reward is the feedback signal; the agent's objective is to maximise the return, the discounted sum of rewards (BOOK Ch.3 p.44). The book also notes that rewards are often **clipped** to a small range to keep learning stable (BOOK Ch.8 p.174).

Every junction, every 5 seconds, pays a **cost** that is the weighted sum of five things; the reward is its negative:

REWARD OF ONE JUNCTION FOR ONE STEP

$$r = -0.1 \times \min\left(Q + 2D + S + 0.5\,P_{\text{own}} + 0.5\,P_{\text{all}} / 98 \;,\; 200 \right)$$

Symbol	Weight	Meaning
Q	1	cars standing still on this junction's incoming lanes (including the feeder roads behind them).
D	2	average cars standing per lane on the roads this junction *feeds into* - so pushing a queue onto a full road is punished (spill-back).
S	1	1 if the junction changed phase in this step, else 0 - a small price on needless switching.
P_{own}	0.5	cars waiting outside the network whose first signal is this junction.
$P_{\text{all}} / 98$	0.5	all cars waiting outside, divided over the 98 junctions - a shared term so every junction cares that the city can take cars in.
min(·, 200), 0.1	-	the cost is capped at 200 (outlier protection) and scaled by 0.1 so rewards sit in roughly [–20, 0] (BOOK Ch.8 p.174).

WORKED EXAMPLE

A junction with 6 standing cars, 2 per lane on the road it feeds, no switch, 4 cars waiting outside for it and 300 waiting outside in total:
cost = 6 + 2·2 + 0 + 0.5·4 + 0.5·300/98 = 6 + 4 + 0 + 2 + 1.53 = 13.53 → reward = –0.1 × 13.53 = **–1.35**. If the next step has 3 standing cars instead of 6, cost falls to 10.5 and the reward rises to –1.05 - the network has been paid for the improvement.

Why a *level*, not a *change*

Earlier versions rewarded the *change* in queue from step to step. That can be gamed: wait while the queue grows (small penalties), release it all at once (one big reward). Paying on the **level** of waiting makes the total return equal to minus the total vehicle-delay, which has no such loophole. The test `tests/test_reward.py` pins this behaviour.

WHERE IN THE CODE

Weights: `src/environment/local_reward.py:28` (`LocalRewardConfig`); formula: `src/environment/local_reward.py:85` (`compute`); clip: `src/environment/local_reward.py:97`; outside-cars bookkeeping: `src/environment/local_reward.py:57`. Values come from `reward:` in `configs/environment.yaml:29`.

7 The learning algorithm: PPO, used as IPPO

7.1 The ladder of ideas (all in the book)

Policy gradient
Nudge the policy so actions that gave a high return become more likely.
BOOK Ch.11 p.225-226

→

Actor-critic
A second network (critic) predicts value; the actor learns from the *advantage* A = Q − V: "better or worse than expected?"
BOOK Ch.10 p.209-211

→

TRPO
Limit how far one update may move the policy, or learning collapses.
BOOK Ch.11 p.240

→

PPO
Same safety idea, but simply *clip* the update. Easy and stable.
BOOK Ch.11 p.245-247

7.2 The PPO objective

PPO CLIPPED OBJECTIVE (BOOK CH.11 P.245-247)

$r_t(\theta) = \pi_\theta(a_t|s_t) \,/\, \pi_{old}(a_t|s_t)$ (how much more likely the new policy makes the action)

$L^{CLIP} = \mathrm{E}[\ \min(\ r_t \cdot A_t\ ,\ \mathrm{clip}(r_t,\ 1-\varepsilon,\ 1+\varepsilon) \cdot A_t\ )\ ]$ with $\varepsilon = 0.2$

$L = -L^{CLIP} + c_1 \cdot (V_\theta(s_t) - G_t)^2 - c_2 \cdot H[\pi_\theta]$ with $c_1 = 0.5,\ c_2 = 0.01$

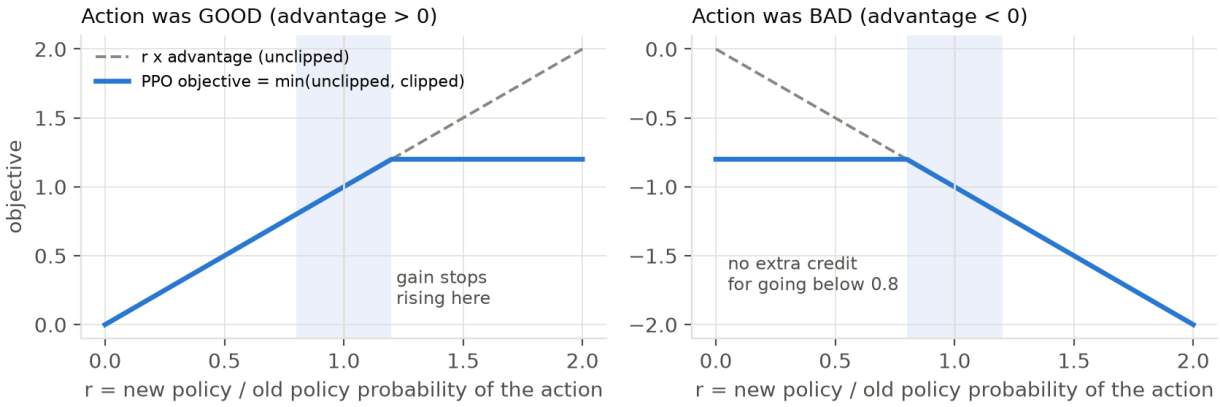


Figure 2 - Why clipping keeps updates small. Left: for a good action (A > 0) the gain stops growing once the action is 20 % more likely than before. Right: for a bad action the push-down also stops at 0.8. Inside the blue band the update behaves normally. (Same picture as the book's BOOK Ch.11 p.246.)

IN PLAIN WORDS

After the network has played some traffic, we look back: "*in this situation I advanced the green and the outcome was better than the critic expected.*" We then make that choice a little more likely - but never by more than 20 % in one update, so one lucky batch cannot wreck what was already learnt. The third term (entropy, BOOK Ch.10 p.211) rewards staying a bit random so the network keeps exploring other options; the book's epsilon-greedy (BOOK Ch.6 p.114) has the same aim, but a policy-gradient method explores by *sampling* from its own probabilities instead.

7.3 The network and the settings

Setting	Value	Meaning
Network (actor and critic each)	22 → 256 → 256 → out, tanh	Two hidden layers of 256 units; tanh activation (BOOK Ch.7 p.132). `configs/ippo.yaml:24`
Optimiser, learning rate	Adam, 3e-4	Gradient descent with adaptive step sizes (BOOK Ch.7 p.136). `configs/ippo.yaml:20`
Discount γ	0.99	Section 3. `configs/ippo.yaml:12`
GAE λ	0.95	Advantage smoothing, section 7.4. `configs/ippo.yaml:13`
Clip range ε	0.2	`configs/ippo.yaml:15`
Value coef. c_1 / entropy coef. c_2	0.5 / 0.01	`configs/ippo.yaml:17`, `configs/ippo.yaml:16`
Samples per update	105,840	180 steps × 98 junctions × 6 simulations. `configs/ippo.yaml:7`
Mini-batches, epochs	8 of 13,230; 10 epochs	Each batch of experience is reused 10 times. `configs/ippo.yaml:8`
Updates	100 (+10 or +20)	10,584,000 steps in stage 1 (`configs/ippo.yaml:6`); stage 2 in section 8.

7.4 Three things that are not in the book

BEYOND THE BOOK

1. **One network for every junction - IPPO (parameter sharing).** "Independent PPO" means each junction decides on its own from its own observation, but all junctions use *the same weights*, and every junction's experience trains them. Per update the one network learns from 98 junctions × 6 simulations = **588 simultaneous "players"**, i.e. 98 times more experience per simulated hour than a network for one junction. It works because junctions play the same role; the active-slot and number-of-greens inputs let the network tell junction types apart. (The book's A3C BOOK Ch.10 p.209 also gathers experience from many parallel copies of the environment; ours is the synchronous variant - all 6 simulations finish a batch, then one update runs.)
2. **Generalised advantage estimation (GAE).** The book's advantage is "return − value" (BOOK Ch.10 p.209). GAE averages short and long look-aheads with weight λ = 0.95: less noise than waiting for the full return, less bias than trusting the critic after one step. Standard in modern PPO libraries (Stable-Baselines3 here).
3. **Split gradient clipping** - our own fix, next section.

WHERE IN THE CODE

Parallel environments: `src/agents/ippo_agent.py:37` (`make_ippo_env`). Model construction with every hyper-parameter above: `src/agents/ippo_agent.py:221` (`build_model`), learning loop `src/agents/ippo_agent.py:292` (`_run_learn`), resume after interruption `src/agents/ippo_agent.py:270`. Entry points: `scripts/run_ippo_training.py:33` (stage 1), `scripts/finetune_ippo.py:18` (stage 2). The PPO implementation itself is Stable-Baselines3's; the contract that a model must be evaluated under the settings it was trained with is `src/agents/model_contract.py:115`.

8 What we changed to beat the baseline - five fixes in plain English

The first versions of the agent *lost* clearly to the baseline (lag 363-476 s against 277 s). We went looking for the reasons one by one; each row is a root cause that was found with evidence and fixed.

#	What was wrong	Fix (plain English)	Code
1	**The network was barely learning.** The policy moved ~0 per update ("clip fraction" = 0).	PPO clips the gradient to keep updates stable - but the actor and critic were clipped *together*. The critic's gradient was ~3,830, the actor's only ~0.19, so the common shrink factor made the actor's steps ~7,600 times smaller. **Fix: clip them separately.** (Figure 3)	`src/agents/split_clip.py:28` `tests/test_split_clip.py`
2	**A see-saw reward.** It paid for *changes* in queue.	Paying for change can be gamed (let it grow, then release). **Fix: pay for the level** of waiting, so return = minus total delay.	`src/environment/local_reward.py:85`
3	**Dead or stuck inputs, and blind to timing.** Two inputs sat at their maximum 81 % and 90 % of the hour; the signal the baseline uses (time in phase vs its minimum) was missing.	**Fix: a new, smaller 22-number observation** with the per-phase timing ratios and a demand number per green (section 4).	`src/environment/local_observation.py:151`
4	**Blind behind short lanes.** A 20 m stop-line lane hid 60 queued cars.	**Fix: look up to 300 m upstream** and count halted cars on the unsignalised feeder roads.	`src/environment/local_observation.py:191`
5	**Blind to cars waiting outside the network.** After stages 1-4 the model was already ~26 s of lag ahead but lost ~100 more cars at the network entrance than the baseline.	**Fix, stage 2:** add outside-waiting cars to the demand of the lane they will queue at, then fine-tune the trained network for 10 (A) or 20 (B) more updates.	`configs/environment_pending.yaml:16` `scripts/finetune_ippo.py:18`

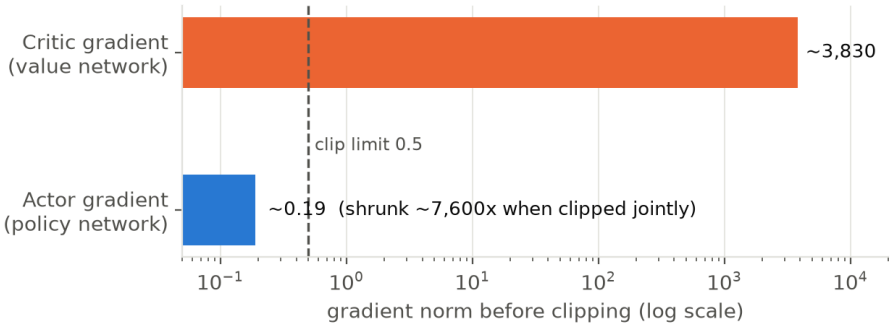


Figure 3 - Fix 1. Typical gradient sizes during stage-1 training. With one shared limit of 0.5 the critic's huge gradient sets the shrink factor for both networks, so the actor's update was reduced about 7,600x. With separate limits each network is clipped on its own.

HOW SPLIT CLIPPING WORKS (ABOUT 40 LINES)

PyTorch's `clip_grad_norm_` is temporarily replaced by a version that clips the actor's parameters and the critic's parameters as two groups (`src/agents/split_clip.py:28`); `SplitClipPPO` (`src/agents/split_clip.py:62`) switches it on during `train()` and logs both norms. It is enabled by `split_clip: true` in `configs/ippo.yaml:19` .

BEYOND THE BOOK

Gradient clipping against "exploding gradients" is mentioned in the book only in passing (e.g. BOOK Ch.7 p.149); clipping *two networks separately inside PPO* is our own engineering fix, validated by the change in learning behaviour and by the final results, not by a reference.

The two stages of training

- **Stage 1** - from scratch, 100 PPO updates (10,584,000 steps, about 7 hours), recipe `configs/ippo.yaml` + `configs/environment.yaml`.
- **Stage 2** - continue the stage-1 network with `pending_in_demand: true` (`configs/ippo_finetune.yaml` + `configs/environment_pending.yaml`). The input size is unchanged, so the trained weights are reused; only what feeds the demand numbers changes. Model A: +10 updates (seed 42). Model B: +20 updates (seed 1042).

---

9 How we measure - a fair test

9.1 Same traffic for model and baseline

SUMO has randomness (exact departure moments, driver behaviour). A **seed** fixes that randomness. For each of **10 seeds** (7, 11, 19, 23, 31, 42, 57, 91, 123, 777) we run baseline, fixed-time, model A and model B on *exactly the same traffic* and compare run against run ("paired by seed"). 4 controllers × 10 seeds = 40 one-hour simulations. The seeds used while training were all above 100,000 (each parallel training worker adds 100,000 × its number to the seed, `src/environment/sumo_session.py:78`), so none of the ten test seeds was ever seen in training.

9.2 The main number: lag per scheduled car

LAG OF ONE SCHEDULED CAR (W = 08:00)

$$\text{lag} = (\,W - t_{\text{ideal}}\,)^{+} - (\,W - t_{\text{arrival}}\,)^{+} \quad \text{with } t_{\text{arrival}} := W \text{ if the car never finished or never entered}$$

$$\textbf{lag per scheduled car} = \text{average of lag over } \textbf{all} \sim 15{,}000 \text{ scheduled cars}$$

WORKED EXAMPLE

A car would finish at 07:30 on an empty road (t_{ideal}). Case 1: it really finishes at 07:42 → lag = 30 min − 18 min = **12 min**. Case 2: it never finishes (still queuing at 08:00, or stuck outside) → lag = 30 min − 0 = **30 min**: it is charged for every minute until the window closed. So a controller cannot look good by letting some cars simply disappear from the statistics - the usual weakness of "mean trip time of the cars that finished".

9.3 The other measures

- **Never inserted** - scheduled cars that could not enter the network because the entry road was full. **Depart delay** - waiting time at the entrance for those that did enter.
- **Backlog** - cars still in the network (or waiting outside) at 08:00. **Due completion** - share of cars that should have finished by 08:00 that did. **Overdue cars** - how many were due but not finished.

- **Waiting time, queue, speed, time loss, trip duration, throughput, trips completed** - the classic traffic measures. **Teleports** - SUMO removes cars stuck for too long: a sign of gridlock.

9.4 When do we say "beaten"?

We take the 10 per-seed differences (model − baseline) and require **both**: (i) the 95 % confidence interval of the mean difference excludes zero on the good side, and (ii) the exact **sign test** is significant - the model wins on at least 9 of 10 seeds (chance of 10 / 10 by luck = $2 \times 0.5^{10} = 0.002$). One strange seed therefore cannot carry a verdict. The table in section 10 shows the verdict for each measure; weaker outcomes are named honestly ("better, not significant", "level").

> **WHERE IN THE CODE**
>
> - Scoring of one run (lag formula at line 420): `src/evaluation/drain_metrics.py:387` (`score_run`); scheduled demand and the empty-road ideal time: `src/evaluation/drain_metrics.py:299`.
> - Running a controller on a seed: `src/evaluation/evaluate.py:77`; command-line: `scripts/evaluate.py:23`.
> - Paired statistics and verdict rule: `scripts/report_eval.py:67`, verdict at `scripts/report_eval.py:87`; interval helpers `src/evaluation/statistics.py`. The full result table is in `docs/TEST_REPORT_10_SEEDS.md`.

10 Results

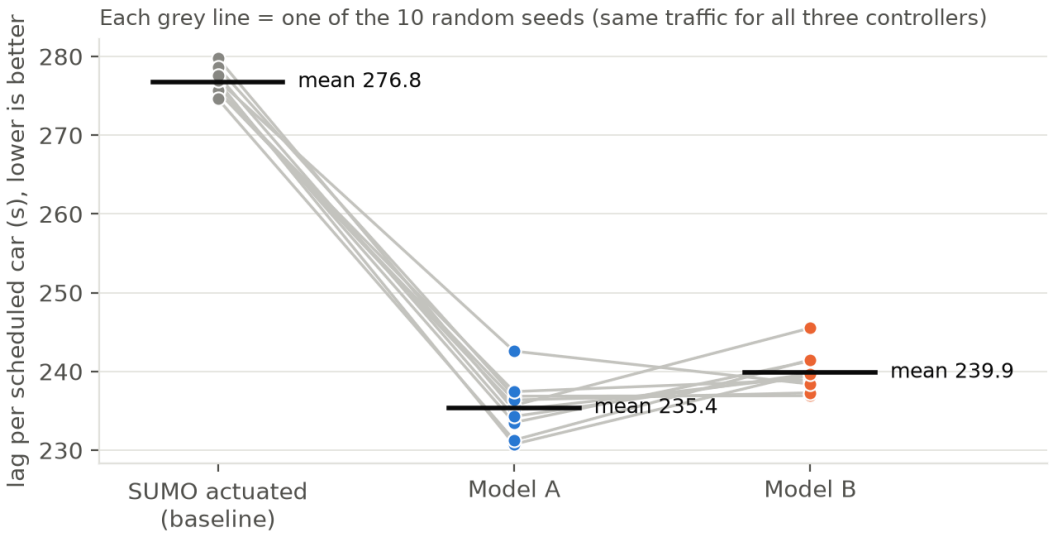


Figure 4 - Lag per scheduled car on each of the 10 seeds. Every seed's line goes down from the baseline to model A, and from the baseline to model B: the lead is not an accident of one seed.

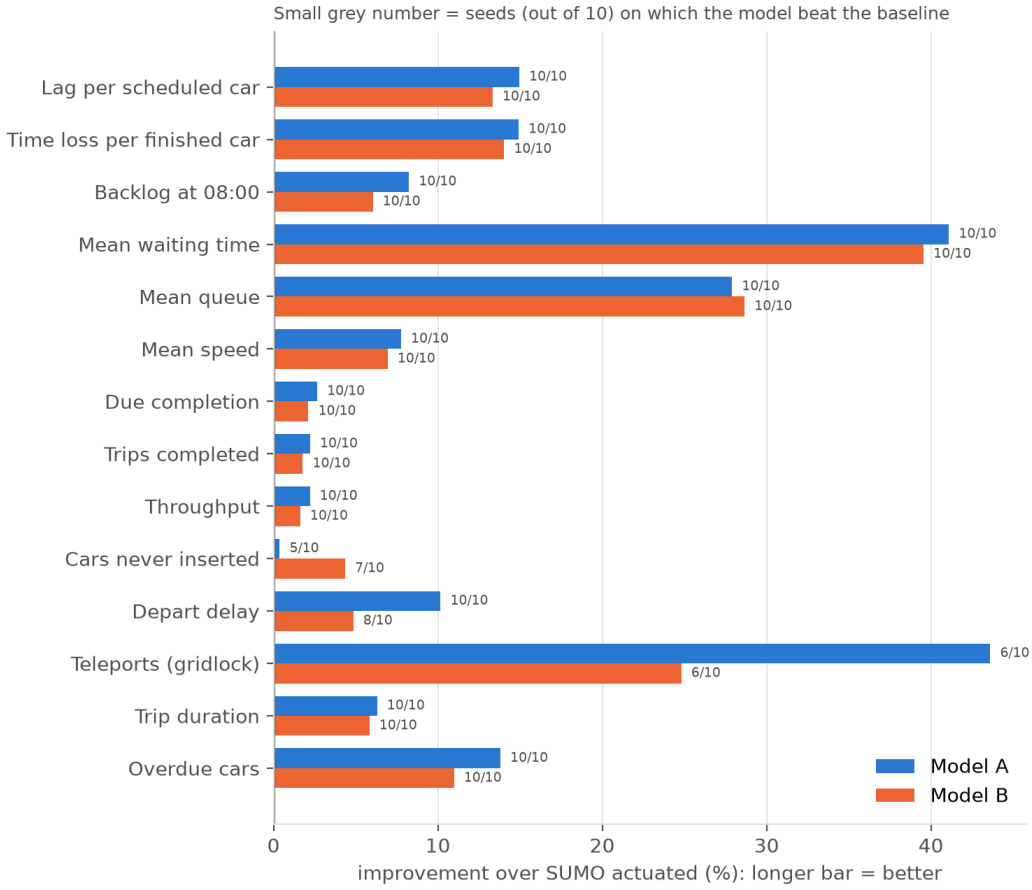


Figure 5 - Improvement over the baseline on all 14 measures (longer bar = better). Two bars are not wins: *cars never inserted* is level, and *teleports* looks large but varies too much between seeds to claim.

Measure (mean of 10 seeds)	SUMO actuated	Model A	Model B	Verdict A / B
Lag per scheduled car, s ↓	276.8	235.4	239.9	beaten / beaten
Time loss per finished car, s ↓	336.6	286.4	289.4	beaten / beaten
Backlog at 08:00, cars ↓	3,108	2,852	2,921	beaten / beaten
Mean waiting time, s ↓	6.39	3.77	3.86	beaten / beaten
Mean queue ↓	0.440	0.318	0.314	beaten / beaten
Mean speed, m/s ↑	8.70	9.37	9.30	beaten / beaten
Due completion ↑	0.839	0.861	0.856	beaten / beaten
Trips completed ↑	11,585	11,842	11,791	beaten / beaten
Throughput (arrived / departed) ↑	0.796	0.813	0.809	beaten / beaten
Trip duration, s ↓	745	699	702	beaten / beaten
Overdue cars ↓	1,853	1,597	1,649	beaten / beaten
Depart delay, s ↓	15.7	14.1	14.9	beaten (10/10) / better, interval only (8/10)
Cars never inserted ↓	503	501	481	better, not significant / better, not significant
Teleports (gridlock) ↓	13.3	7.5	10.0	better (interval only) / better, not significant

Score: model A **beaten on 12 of 14**, model B **on 11 of 14**; the other measures are better on average but not significantly; **none is worse**. Held-out windows that were never trained on (one seed, model A only): 08:00-09:00 lag 230.7 → 177.8 s; 17:00-18:00 lag 108.5 → 50.0 s.

11 Honest limits - what this does not show

PLEASE READ BEFORE QUOTING A RESULT

- **"Cars never inserted" is not won.** A first 3-seed test suggested a win (473 vs 539); the 10-seed test shows the baseline was unusually high on those three seeds. The honest statement: the stage-2 fix *closed* the entrance gap (stage 1 lost ~100 more cars than the baseline) but did not open a lead.
- **Teleports** are too noisy (6 / 10 seeds) to claim in either direction.
- **One scenario.** Training and test use the same network and the same demand pattern (07:00-08:00 InTAS); seeds only change the random details. The 08:00 and 17:00 windows (one seed, model A) are a first check on other times of day; a different city or real traffic is untested.
- **Sensing is generous.** The agent reads exact halted-car counts per lane, including cars still waiting outside. Real junctions have detectors with limited range and noise; degraded sensing was not tested.
- **Choices made after seeing data.** Model B received a second fine-tune block (10 → 20 updates) after a 3-seed result, and the strict "beaten" rule was fixed after the first results (it is stricter than the first rule). Both are disclosed; neither was selected on the 10-seed test set.
- **More phase changes** than the baseline (section 5) - the results already include this cost, but wear and comfort are not modelled.

12 Reproduce it

Step	Command (from the repository root)
Set up	`make install && make sumo && source .env.sumo` (Python 3.11, Eclipse SUMO 1.20+, InTAS fetched at setup)
Tests (no SUMO needed)	`make test lint`
Stage 1 training (~7 h)	`make train` → `scripts/run_ippo_training.py:33` with `configs/ippo.yaml` + `configs/environment.yaml`
Stage 2 fine-tune	`python scripts/finetune_ippo.py --init-model <stage-1 model> ... --ppo-config configs/ippo_finetune.yaml --env-config configs/environment_pending.yaml`
Evaluate one run	`python scripts/evaluate.py --controller ippo --model <model_final.zip> --out artifacts/eval/ippo`
10-seed paired report	`python scripts/report_eval.py --root artifacts/eval10_run --baseline sumo_actuated --models ippo_A ippo_B --extra fixed_time`

Trained weights are not stored in git; hashes and where to find them are in `docs/MODELS.md` and `artifacts/models/MANIFEST.json`. Every result file records the code version and settings it was produced with.

13 One-paragraph summary for the viva

> **IF ASKED "WHAT DID YOU DO?"**
>
> We modelled morning traffic in Ingolstadt as a reinforcement-learning problem: one shared neural network, copied to all 98 junctions, sees 22 numbers about its junction and chooses every 5 seconds to keep or advance the green, behind a safety layer; it is paid minus the cars waiting. We trained it with PPO (book Ch.11), found why the first versions failed - the actor's learning signal was being squashed by the critic's gradient, the reward could be gamed, the inputs were blind to the baseline's timing logic, to long queues behind short lanes and to cars waiting outside - fixed each, and then tested on 10 seeds of identical traffic against SUMO's own actuated control. The trained network is significantly better on 12 (and 11) of 14 measures - e.g. 41 % less waiting, about 200-260 more completed trips per hour - and worse on none; it does not reduce the number of cars blocked at the network entrance, and has only been tested on this one scenario.